

2 Ejercicios: Input System + Transform

Dos ejercicios nuevos, sencillos y sin Rigidbody ni CharacterController. El objetivo es practicar entrada y movimiento directo.

Objetivo

Contexto: ya has practicado un cubo con movimiento, inputs y una camara en tercera persona al hombro. Estos ejercicios eliminan la camara compleja y cambian la forma de interactuar para practicar otras variantes del Input System.

Reglas de ambos ejercicios

Solo se permite Input System para leer las acciones y Transform para mover el objeto. No uses Rigidbody, CharacterController, NavMesh ni fisica. La gracia es dominar la relacion InputAction -> codigo -> transform.

Ejercicio 1 - Drone imantado con raton

Idea: el jugador controla un pequeno drone visto desde arriba. Manteniendo el boton izquierdo del raton, el drone se mueve hacia el punto del suelo donde esta el cursor. Al soltar el boton, el drone se queda quieto. No hay tercera persona ni joystick de camara.

Aprendizajes

- Leer una accion de tipo Button. - Convertir la posicion del raton en un punto del mundo. - Mover con Transform sin fisicas. - Separar lectura de input y movimiento.

Montaje

Elemento	Configuracion
Scene	Plane o suelo, una camara fija mirando desde arriba y un Cube/Sphere como Drone.
Input Actions	Action Map Gameplay -> Action MoveToCursor de tipo Button -> binding /leftButton.
Camara	Ejemplo: (0, 12, 0), rotacion (90, 0, 0) si usas el sistema de coordenadas que prefieras. Ajusta segun tu escena.
Movimiento	Usar transform.position y Vector3.MoveTowards. Sin Rigidbody.

DroneMagnet.cs

```
using UnityEngine;
using UnityEngine.InputSystem;

public class DroneMagnet : MonoBehaviour
{
    [SerializeField] private Camera sceneCamera;
    [SerializeField] private float moveSpeed = 6f;
    [SerializeField] private InputActionReference moveToCursor;

    private void OnEnable()
    {
        moveToCursor.action.Enable();
    }

    private void OnDisable()
    {
        moveToCursor.action.Disable();
    }

    private void Update()
    {
        if (!moveToCursor.action.IsPressed()) return;

        Vector2 mouse = Mouse.current.position.ReadValue();
        Ray ray = sceneCamera.ScreenPointToRay(mouse);
        Plane ground = new Plane(Vector3.up, Vector3.zero);

        if (ground.Raycast(ray, out float distance))
        {
            Vector3 target = ray.GetPoint(distance);
            transform.position = Vector3.MoveTowards(
                transform.position,
                target,
                moveSpeed * Time.deltaTime
            );
        }
    }
}
```

Importante: el código sigue usando Input System. `Mouse.current.position` se usa para obtener la posición del cursor, mientras que la acción configurada controla si el movimiento está activo.

Variaciones para subir un nivel

Haz que el dron tenga dos velocidades: lenta al pulsar el botón y rápida si mantienes Shift. Otra variante es limitar el movimiento a un rectángulo del escenario.

Ejercicio 2 - Saltimbanqui magnetico por carriles

Idea: controla una criatura que se desplaza por un circuito de tres carriles. W/S avanza y retrocede; A/D cambia de carril. La criatura no gira una cámara: el objetivo es que el desplazamiento se sienta como una máquina de arcade. Todo se hace cambiando directamente Transform.

Aprendizajes

- Crear un Vector2 de movimiento a partir de varias acciones.
- Usar un único Action Map.
- Normalizar el vector para evitar que diagonal sea más rápida.
- Mover con `Transform.Translate` o `transform.position`.

Input Actions sugeridas

Accion	Tipo	Bindings
Move	Value / Vector2	2D Vector Composite: W/A/S/D o flechas
Sprint	Button	/space

Montaje

Crea una plataforma con tres posiciones X: -3, 0, 3. La criatura se mueve en X y Z. La altura Y no cambia. El objetivo no es crear un runner completo, sino practicar la conversion del input a movimiento y una velocidad modificada por otro input.

ArcadeRunner.cs

```
using UnityEngine;
using UnityEngine.InputSystem;

public class ArcadeRunner : MonoBehaviour
{
    [SerializeField] private InputActionReference move;
    [SerializeField] private InputActionReference sprint;
    [SerializeField] private float speed = 4f;
    [SerializeField] private float sprintMultiplier = 1.8f;

    private void OnEnable()
    {
        move.action.Enable();
        sprint.action.Enable();
    }

    private void OnDisable()
    {
        move.action.Disable();
        sprint.action.Disable();
    }

    private void Update()
    {
        Vector2 input = move.action.ReadValue<Vector2>();
        if (input.sqrMagnitude > 1f)
            input.Normalize();

        float currentSpeed = sprint.action.IsPressed()
            ? speed * sprintMultiplier
            : speed;

        Vector3 direction = new Vector3(input.x, 0f, input.y);
        transform.position += direction * currentSpeed * Time.deltaTime;
    }
}
```

Que debes comprobar

Prueba	Resultado esperado
W	Avanza en Z.
S	Retrocede en Z.
A / D	Se mueve lateralmente por los carriles.
W + A	Avanza en diagonal sin duplicar la velocidad.
Space + direccion	Se mueve mas rapido mientras Space esta pulsado.
Sin Rigidbody	El objeto se sigue moviendo porque todo pasa por Transform.

Reto final

Haz que el cambio de carril se sienta mas arcade: en lugar de permitir cualquier X, redondea la posicion a uno de los tres carriles. Sigue evitando fisicas; el reto debe resolverse solo con Input System y Transform.

Checklist de entrega

1. Input Actions creadas y habilitadas. 2. Los bindings responden. 3. No hay Rigidbody ni CharacterController. 4. El movimiento usa Transform. 5. El movimiento multiplica por Time.deltaTime. 6. El movimiento diagonal no es mas rapido. 7. Los valores principales se pueden ajustar desde el Inspector.